R: limpieza, manipulación y exploración de datos

3er encuentro: Carga de datos y gráficos sencillos

Jonatán Perren

14 de abril de 2026

Este eje proporciona las herramientas necesarias para trabajar con los formatos de archivo más comunes en el ámbito profesional y académico.

Abordaremos la lectura de archivos de Excel (formato propietario ampliamente utilizado), archivos de texto delimitado (CSV, TSV) y otros formatos estructurados.

Veremos cómo gestionar proyectos de RStudio mediante archivos .Rproj, estableciendo flujos de trabajo organizados con rutas relativas que favorecen la reproducibilidad.

Revisaremos algunas buenas prácticas para la inspección inicial de los datos importados, verificando tipos de columnas, dimensiones y detección temprana de problemas de codificación o formato.

Uno de los problemas más comunes al comenzar a programar es la gestión de los archivos. ¿Dónde están mis datos? ¿Dónde se guardan mis gráficos? ¿Por qué mi código no corre en la computadora de mi compañeroooo? La respuesta suele estar en el manejo del directorio de trabajo y los proyectos .

¿Qué es y cómo configurar el directorio de trabajo?

El directorio de trabajo (Working Directory) es la carpeta donde R «mira» por defecto. Si le pedís leer un archivo **`datos.csv`**, R lo buscará en esa carpeta. Si guardás un gráfico, se guardará ahí.

Podés saber en qué carpeta estás «parado» con el comando

```
getwd() # "get working directory"
```

Y podés cambiar esa carpeta con

```
setwd("C:/Usuarios/MiUsuario/Documentos/MiCursoR") # "set working directory"
```

Usar `setwd()` con rutas absolutas (como la de arriba) es una mala práctica porque hace que tu código sea frágil. Si movés la carpeta, le cambiás el nombre, o le pasás el script a otra persona con otro usuario, el código fallará porque la ruta `"C:/Usuarios/MiUsuario ... "` no existirá.

Para evitar esto, tenemos los proyectos.

Proyectos

Para solucionar esto, RStudio introdujo los R Projects (`.Rproj`). Un proyecto es un archivo que define la raíz de tu trabajo. Cuando abres un proyecto, RStudio automáticamente configura el directorio de trabajo en la carpeta donde está ese archivo.

Pasos para crear un proyecto

1. Andá a `File > New Project`.
2. Elegí `New Directory` para una carpeta nueva o `Existing Directory` si ya tenés la carpeta preparada.
3. Elegí `New Project`.
4. Dale un nombre a la carpeta y elegí dónde guardarla.

Al abrir este proyecto, `getwd()` siempre apuntará a la carpeta del proyecto. Esto hace que tu trabajo sea portable.

Rutas relativas

Exacto, tener el proyecto no es suficiente si seguimos siendo desordenados. Para aprovechar la portabilidad, debemos usar rutas relativas y una estructura de carpetas consistente.

Estructura recomendada

Dentro de tu carpeta de proyecto, crea subcarpetas:

- `datos/`: Acá guardá tus excels, csv, etc.
- `scripts/`: Acá guardá tus archivos `.R` o `.qmd`.
- `resultados/`: Acá guardá los gráficos o tablas que exportes.

Rutas Relativas

En lugar de decirle a R dónde está el archivo desde el disco C (`"C:/ ... "`), le decís dónde está desde la carpeta del proyecto.

- MALA PRÁCTICA: `"C:/Usuarios/Juan/ProyectoR/datos/tabla.csv"`
- BUENA PRÁCTICA: `"datos/tabla.csv"`

Si usas rutas relativas, puedes mover toda la carpeta del proyecto a cualquier computadora y seguirá funcionando 😉.

Carga de datos

En general, no vamos a trabajar con un entorno cerrado. Siempre necesitaremos datos externos. Los más frecuentes, y que no son datos que vayamos a analizar, son los paquetes.

Los paquetes

Los paquetes son piezas de código empaquetadas que ofrecen funcionalidades que el software de base no implementa o que son engorrosas de escribir. Por ejemplo, la función `lag()` (que me imagino que ya buscaste para qué sirve) no viene implementada por defecto en R, por lo que debe ser incorporada mediante un paquete.

Instalar un paquete

Para instalar un paquete en nuestro sistema ejecutamos `install.packages()` y le pasamos como cadena de texto (entre comillas) el nombre del paquete.

```
install.packages(c("pacman", "tidyverse", "janitor"))
```

No es necesario que esta instrucción vaya al script. Puede ejecutarse desde la consola.

Ahora bien, ahora el/los paquetes estarán instalados en nuestro sistema. Es necesario cargarlos al proyecto. Para cargar un paquete usamos la instrucción `library()` y le pasamos el nombre del paquete como variable (sin comillas).

```
library(tidyverse)
library(janitor)
```

Esto sí debe ir en el script.

pacman

La librería **pacman** es útil si necesitamos cargar paquetes y no sabemos si están instalados o no. En lugar de probar uno por uno, y luego cargar, podemos hacer algo así

```
library(pacman)
pacman::p_load(fixest, modelsummary)
```

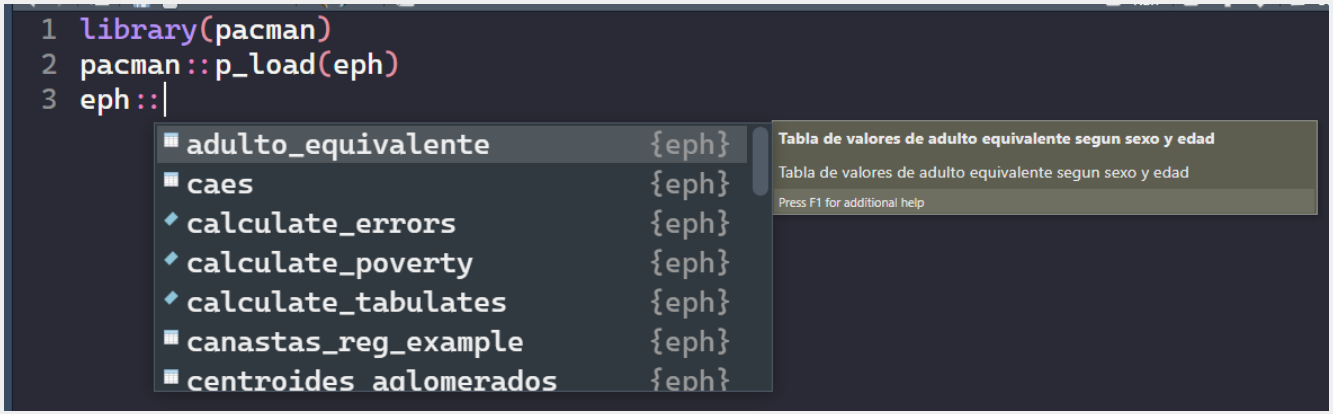
Esto chequea si existe **fixest** y **modelsummary**. Si no existen, los descarga. Luego, carga los paquetes.

¿Y por qué ::?

Si dos paquetes distintos definen una función con el mismo nombre (por ejemplo, la libraría estándar **stats** de R y **dplyr** de Posit tienen una versión propia de **lag**) podemos elegir una u otra con ese prefijo:

```
stats::lag() #lag de R
dplyr::lag() #lag de dplyr
```

También puede ser útil para saber qué funciones, objetos o data frames trae la librería como se puede ver en la siguiente imagen con la **eph**.



Aunque para esto último es mucho más útil ejecutar algo como

```
data(package = "eph") #Intentalo :)
```

Carga de datos

R puede leer casi cualquier formato de datos. Veremos los más importantes. Para esto, utilizaremos principalmente el paquete **`readr`** (incluido en tidyverse) para texto plano, y paquetes específicos para otros formatos.

Recuerda cargar **`tidyverse`** antes de empezar:

```
library(tidyverse)
```

Ficheros .xlsx

Los archivos de Microsoft Excel son el estándar en muchas industrias. Para leerlos, usamos el paquete **`readxl`** (parte del ecosistema **`tidyverse`**, pero a veces se debe cargar explícitamente).

```r
library(readxl)

# Leer la primera hoja del archivo
datos <- read_excel("datos/mi_excel.xlsx")

# Leer una hoja específica (por nombre o índice)
datos_ventas <- read_excel("datos/mi_excel.xlsx", sheet = "Ventas 2023")
```

`read_excel()` detecta automáticamente el tipo de dato de cada columna, pero puedes forzarlo con el argumento **`col_types`** si es necesario.

Ficheros .ods

Los archivos Open Document Spreadsheet (.ods) son la alternativa libre a Excel (usados por LibreOffice, OpenOffice, Google Sheets). Para leerlos, necesitamos el paquete `readODS`.

```r
# install.packages("readODS")
library(readODS)

datos_ods <- read_ods("datos/mi_calc.ods")

# Especificar hoja
datos_hoja3 <- read_ods("datos/mi_calc.ods", sheet = 3)
```

Ficheros .csv

Los archivos CSV (Comma Separated Values) son el formato más estándar para intercambio de datos. Son archivos de texto plano donde las columnas están separadas por comas. No almacenan nada que no sea dato (a diferencia de un fichero .xlsx que guarda colores, formato de texto, etc.).

Usamos la función `read_csv()` del paquete `readr`:

```
datos_csv ← read_csv("datos/archivo.csv")
```

El problema del punto y coma

En países de habla hispana (y Europa), usamos la coma `,` para decimales (ej: 3,14). Por lo tanto, Excel suele guardar los CSV usando punto y coma `;` como separador de columnas para no confundirse. Si tu archivo usa `;` como separador, debés usar `read_csv2()`:

```
# Para CSV con separador ";" y decimal ","
datos_latino ← read_csv2("datos/archivo_latino.csv")
```

Aunque también podés leerlos con `read_csv()`. Averigüá cómo con `?read_csv`.

Ficheros .tsv

Los archivos TSV (Tab Separated Values) son similares a los CSV pero usan el caracter tabulador para separar columnas. Son muy comunes en bioinformática o exportaciones de bases de datos.

```
datos_tsv ← read_tsv("datos/archivo.tsv")
```

Ficheros .rds

El formato `.rds` es un formato binario propio de R. Sirve para guardar **un único objeto** de R (un data frame, una lista, un modelo estadístico, etc.) manteniendo todas sus propiedades intactas (tipos de datos, atributos, etc.).

Es muy útil para guardar resultados intermedios que tardaron mucho en calcularse.

Guardar un objeto

```
saveRDS(mis_datos, "datos_procesados/mis_datos_limpios.rds")
```

Leer un objeto

Acá debemos asignar el resultado a una variable, ya que el archivo no guarda el nombre del objeto, solo su contenido.

```
datos_recuperados <- readRDS("datos_procesados/mis_datos_limpios.rds")
```

Fichero .RData

El formato `.RData` (o `.rda`) también es propio de R, pero permite guardar múltiples objetos a la vez. Es como guardar «todo el espacio de trabajo» o una parte de él.

Guardar objetos

```
save(objeto1, objeto2, file = "resultados/mi_trabajo.RData")
```

Leer objetos

A diferencia de `readRDS`, la función `load()` no se asigna a una variable. Simplemente «carga» los objetos en tu entorno con los nombres que tenían cuando se guardaron.

```
load("resultados/mi_trabajo.RData")
#objeto1 y objeto2 aparecerán en tu panel de Environment.
```

Por lo general, prefiere `.rds` sobre `.RData` para guardar datos, ya que te da control sobre el nombre de la variable al cargarla (`datos ← readRDS( ... )`). El `.RData` puede **sobrescribir** variables existentes sin aviso si tienen el mismo nombre.

Gráficas sencillas

Aunque `ggplot2` es el estándar de oro para visualización en R (y lo veremos más adelante), R base ofrece funciones gráficas muy potentes para realizar un análisis exploratorio de datos rápido. Cuando recibimos un dataset nuevo, queremos ver la forma de los datos rápidamente, sin preocuparnos por colores bonitos o etiquetas perfectas.

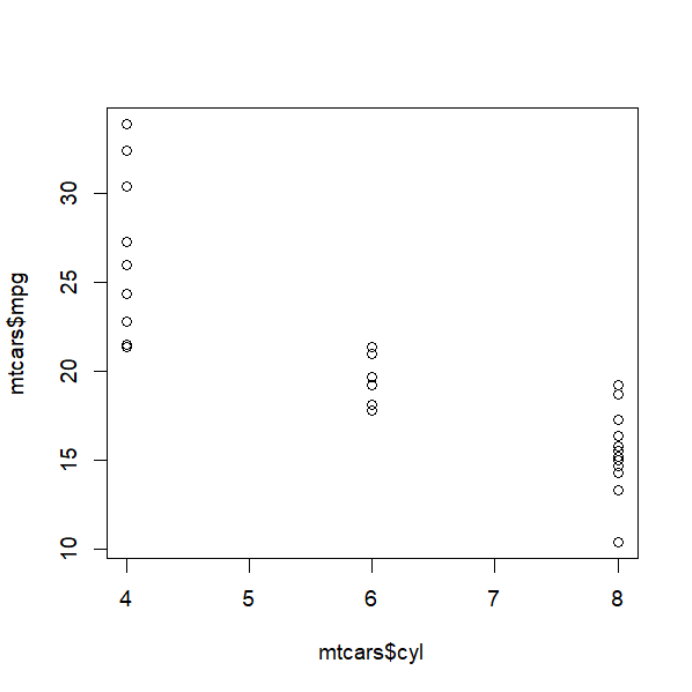
plot

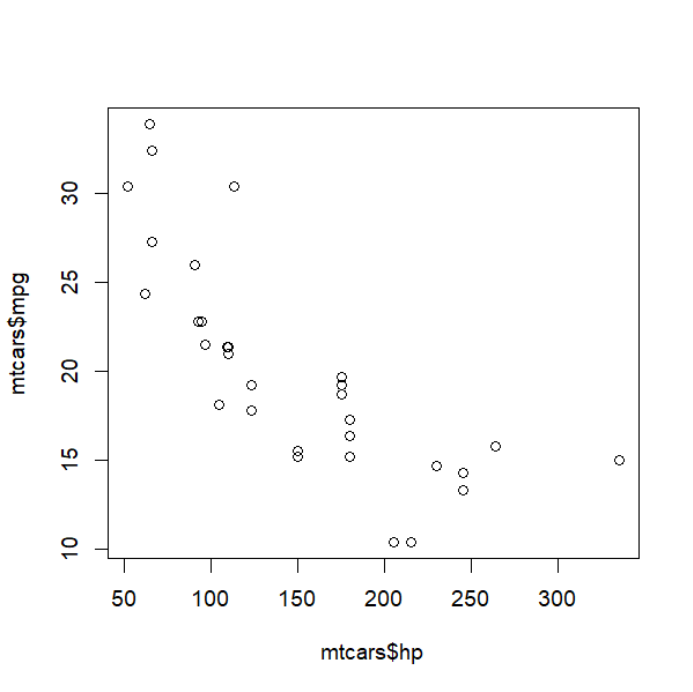
Es una función genérica que cambia su comportamiento según qué datos le pasemos.

- Si le pasamos dos vectores numéricos (x e y), hace un gráfico de dispersión (scatterplot).
- Si le pasamos un factor, hace un gráfico de barras.

```r
# x = Cilindros, y = Millas por galón
plot(mtcars$cyl, mtcars$mpg)

# Relación entre Potencia y Consumo
plot(x = mtcars$hp, y = mtcars$mpg)
```



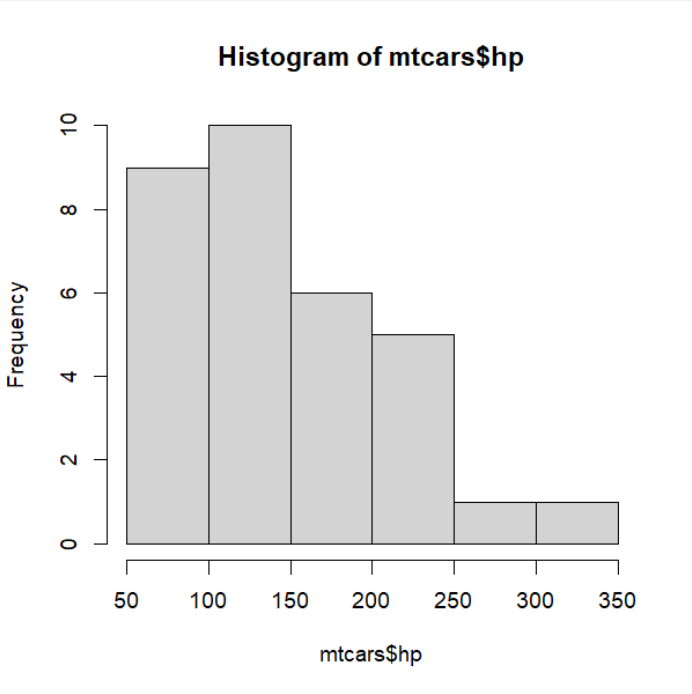


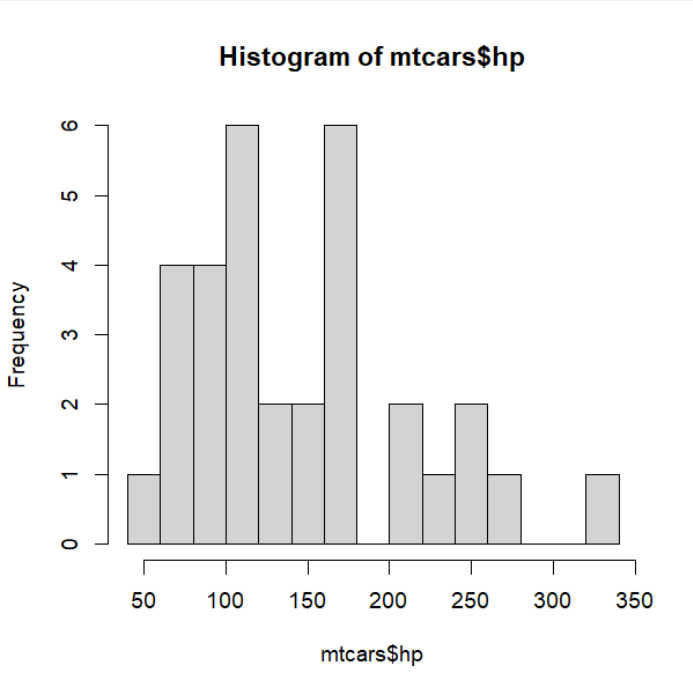
El histograma es fundamental para entender la distribución de una variable numérica continua. Nos dice si los datos están centrados, si son asimétricos o si hay valores inusuales.

```
# Distribución de la potencia de los autos
hist(mtcars$hp)
```

Podemos aumentar o disminuir los cortes con el argumento **`breaks`** para ver más o menos detalle.

```
hist(mtcars$hp, breaks = 20)
```



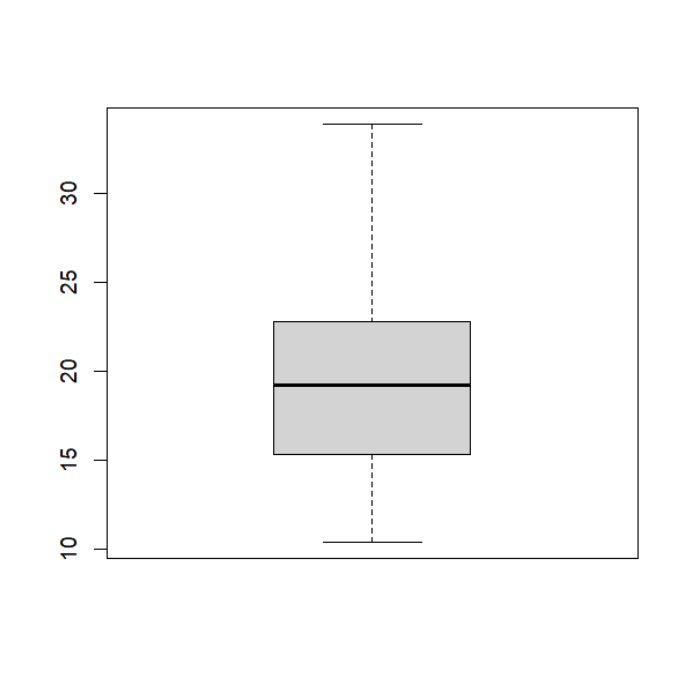


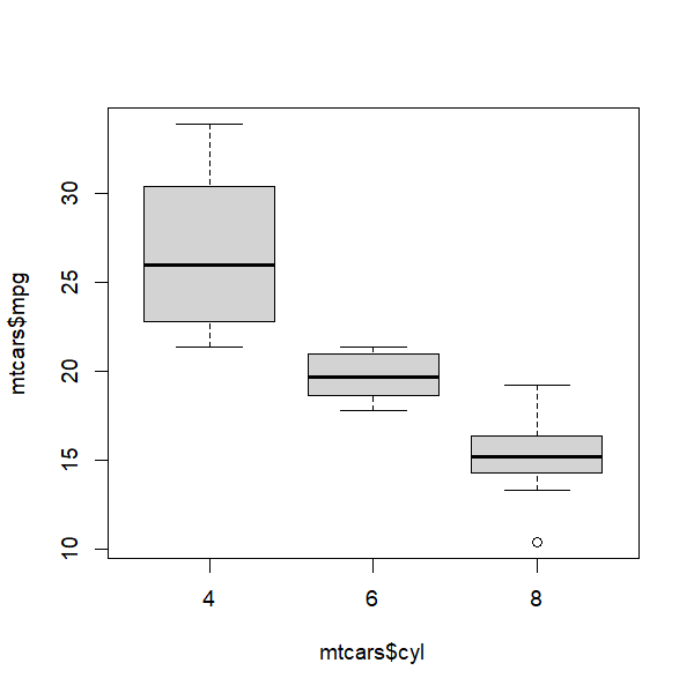
boxplot

El diagrama de caja y bigotes es la mejor herramienta para visualizar la distribución estadística robusta. Nos muestra la mediana (la línea gruesa), los cuartiles (la caja) y los outliers o valores atípicos.

```r
# Boxplot simple de una variable
boxplot(mtcars$mpg)

# Boxplot comparativo: Rendimiento según cantidad de cilindros
# La fórmula se lee: "Millas por galón en función de Cilindros"
boxplot(mtcars$mpg ~ mtcars$cyl)
```





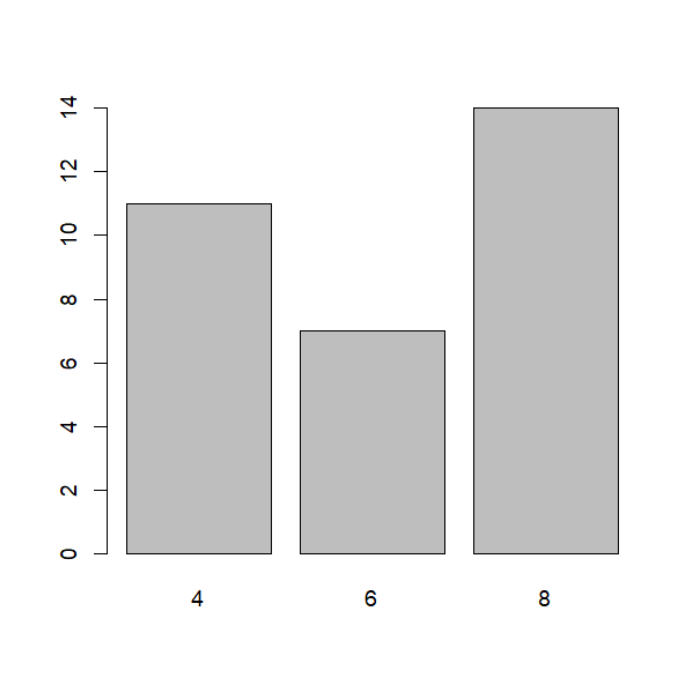
barplot

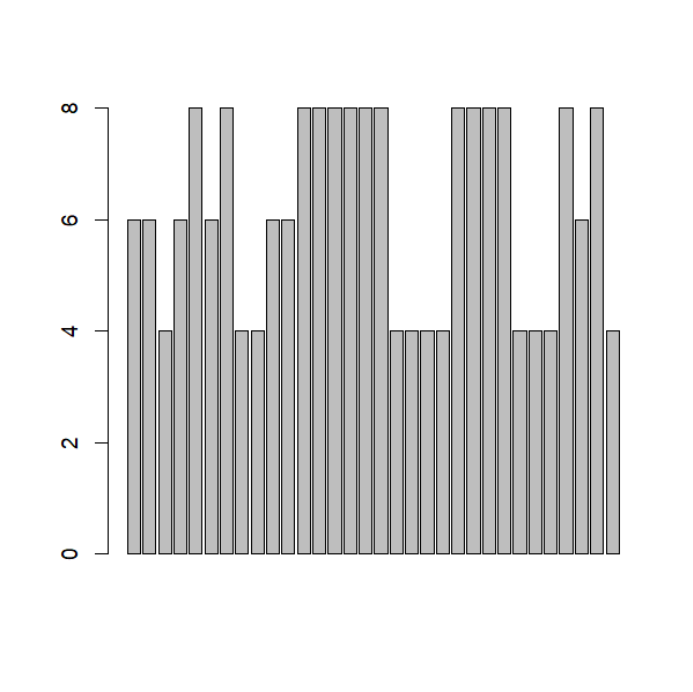
Para variables categóricas (como factores o texto), necesitamos contar cuántas veces aparece cada categoría antes de graficar. **`barplot()`** funciona muy bien en combinación con **`table()`**. ¿Te acordás de que hace **`table()`**, no?

```r
# Primero creamos la tabla de frecuencias
conteos <- table(mtcars$cyl)

# Luego graficamos
barplot(conteos)

barplot(mtcars$cyl) #y esto qué grafica? es útil?
```





Con estas 4 funciones (`plot`, `hist`, `boxplot`, `barplot`) puedes tener una idea muy clara de tus datos en segundos, antes de empezar cualquier análisis complejo.

Actividad integradora semanal (se sube al aula)

1. Buscá un dataset en formato `.csv` o `.xlsx` de tu interés (podés usar portales de datos abiertos como `datos.gob.ar`, `kaggle.com` u otros). Creá un proyecto de RStudio con la estructura recomendada, ubicá el archivo en `datos/` y cargalo con la función apropiada. Realizá una inspección inicial con `dim()`, `head()` y `summary()`. ¿Cuántas filas y columnas tiene? ¿Qué tipo de dato tiene cada columna?
2. Elegí una variable numérica del dataset y graficá su distribución con `hist()`. Probá al menos dos valores distintos para `breaks`. Describí brevemente lo que observás: ¿es simétrica la distribución? ¿hay concentración en algún rango?
3. Elegí una variable numérica y una categórica. Usá `boxplot()` con notación de fórmula para comparar la variable numérica entre los grupos de la categórica. Luego, con `barplot(table( ... ))` mostrá la frecuencia de cada categoría. Comentá lo que encontrás en ambos gráficos.